QuEra Challenge

The Ôperators

Goal:

We will explain how a one qubit gate may be harder to implement than a 2-qubit gate by studying:

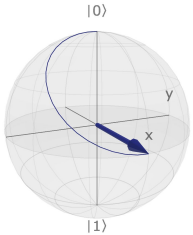
$$R_z(\pi/2^k) = \begin{pmatrix} e^{-i\pi/2^{k+1}} & 0 \\ 0 & e^{i\pi/2^{k+1}} \end{pmatrix} \sim \begin{pmatrix} 1 & 0 \\ 0 & e^{2i\pi/2^{k+1}} \end{pmatrix}$$

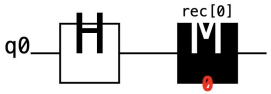
Clifford+T Gates in Bloqade

Hadamard Gate => Superposition State

```
@squin.kernel
def demo_h() -> Measurement:
        qubits = squin.qalloc(1)
        squin.h(qubits[0])
        bits = squin.broadcast.measure(qubits)
        return bits

demo_circuit(demo_h)
```



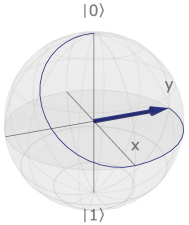


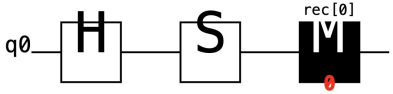
{(<Measurement.Zero: 0>,): 0.516,
(<Measurement.One: 1>,): 0.484}

S Gate => Rotates Z-axis by 180º (add phase to state)

```
@squin.kernel
def demo_s() -> Measurement:
        qubits = squin.qalloc(1)
        squin.h(qubits[0])
        squin.s(qubits[0])
        bits = squin.broadcast.measure(qubits)
        return bits

demo_circuit(demo_s)
```





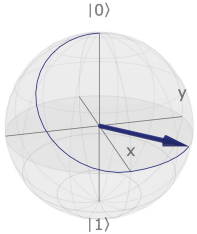
{(<Measurement.Zero: 0>,): 0.5,
(<Measurement.One: 1>,): 0.5}

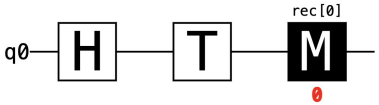
Clifford+T Gates in Bloqade

T Gate => Rotates Z by 90º (add phase to state)

```
@squin.kernel
def demo_t() -> Measurement:
        qubits = squin.qalloc(1)
        squin.h(qubits[0])
        squin.t(qubits[0])
        bits = squin.broadcast.measure(qubits)
        return bits

demo_circuit(demo_s)
```



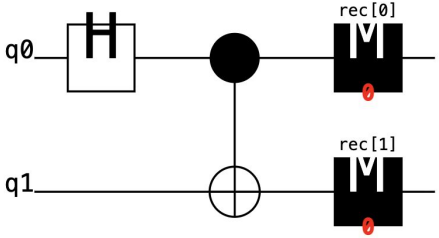


{(<Measurement.Zero: 0>,): 0.509,
(<Measurement.One: 1>,): 0.491}

CNOT Gate (+2 Qubits) => Flips state of 1 Qubit (If control Qubit is 1)

```
@squin.kernel
def demo_cx() -> Measurement:
        qubits = squin.qalloc(2)
        squin.h(qubits[0])
        squin.cx(qubits[0], qubits[1])
        bits = squin.broadcast.measure(qubits)
        return bits

demo_circuit(demo_cx)
```



{(<Measurement.One: 1>, <Measurement.One: 1>): 0.5,
(<Measurement.Zero: 0>, <Measurement.Zero: 0>): 0.5}

Simple rotations

$$k = 0: \quad R_z(\pi) = \begin{pmatrix} e^{-i\pi/2} & 0 \\ 0 & e^{i\pi/2} \end{pmatrix} = \begin{pmatrix} -i & 0 \\ 0 & i \end{pmatrix} = Z = S^2 = T^4$$

$$k = 1: \quad R_z(\pi/2) = \begin{pmatrix} e^{-i\pi/4} & 0 \\ 0 & e^{i\pi/4} \end{pmatrix} = S = T^2$$

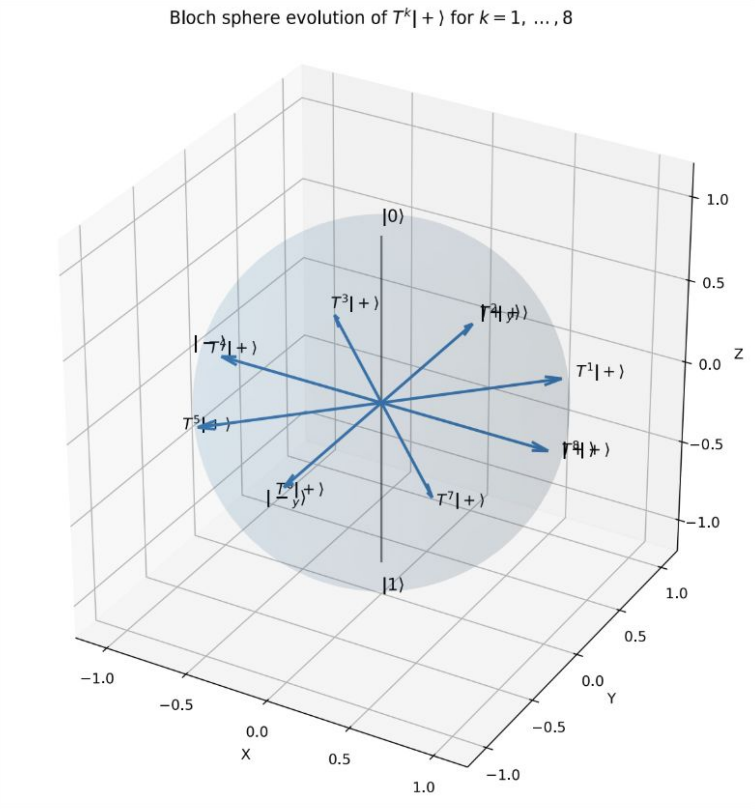
$$k = 2: \quad R_z(\pi/4) = \begin{pmatrix} e^{-i\pi/8} & 0 \\ 0 & e^{i\pi/8} \end{pmatrix}$$

Not so simple rotations

The S and T gates generate the finite *abelian* group of pizza slice rotations.

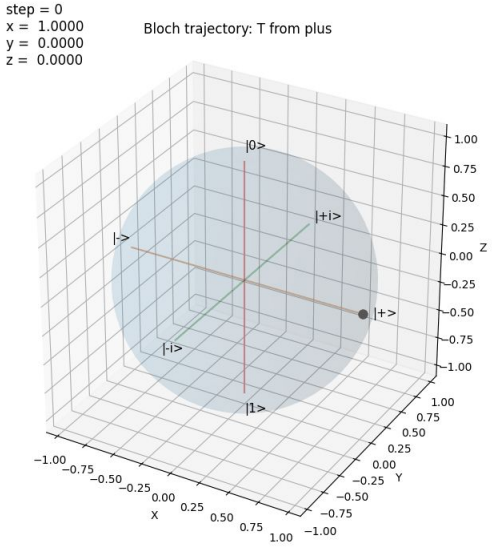
Evidently we need the Hadamard gate to break out!

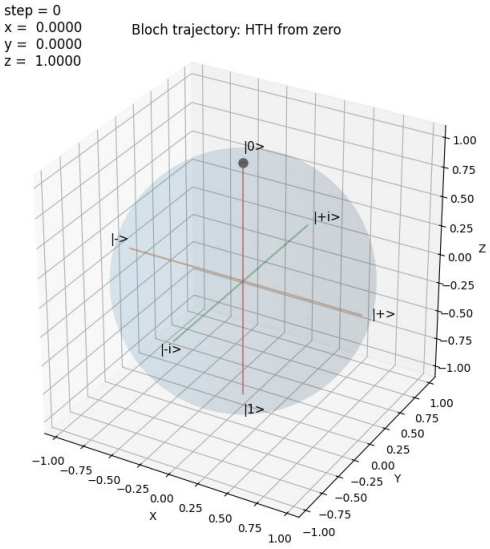
Note that the Hadamard gate does *not* commute with S and T

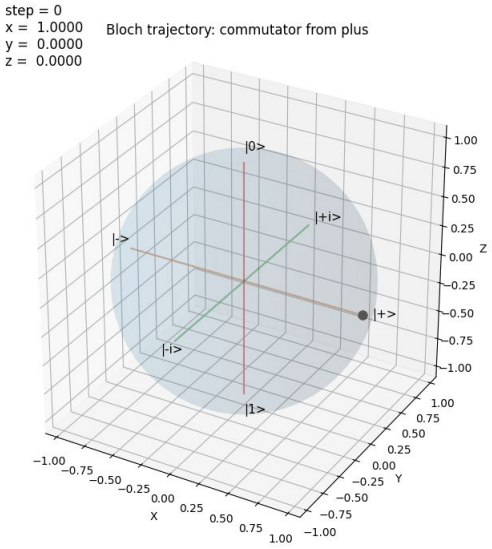


Visualization of rotations generated by the T-gate acting on the $|+\rangle$ state.

Commutating

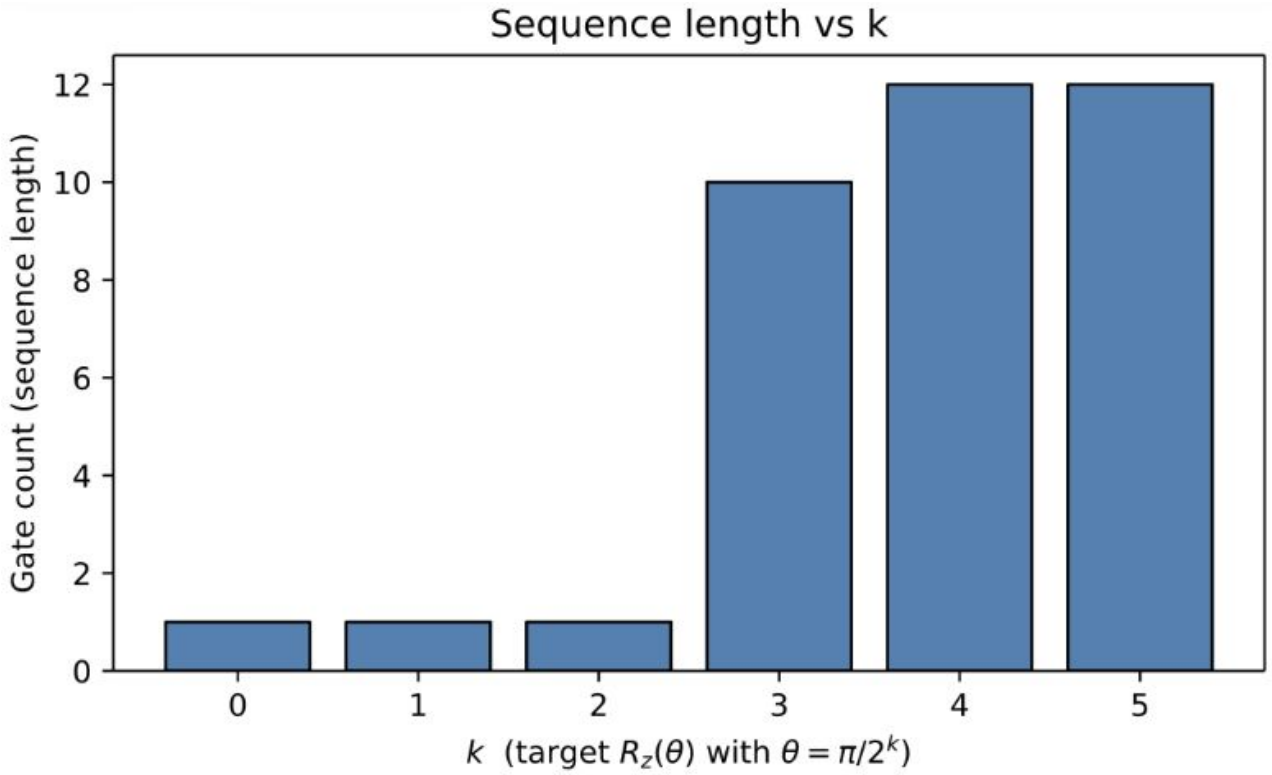






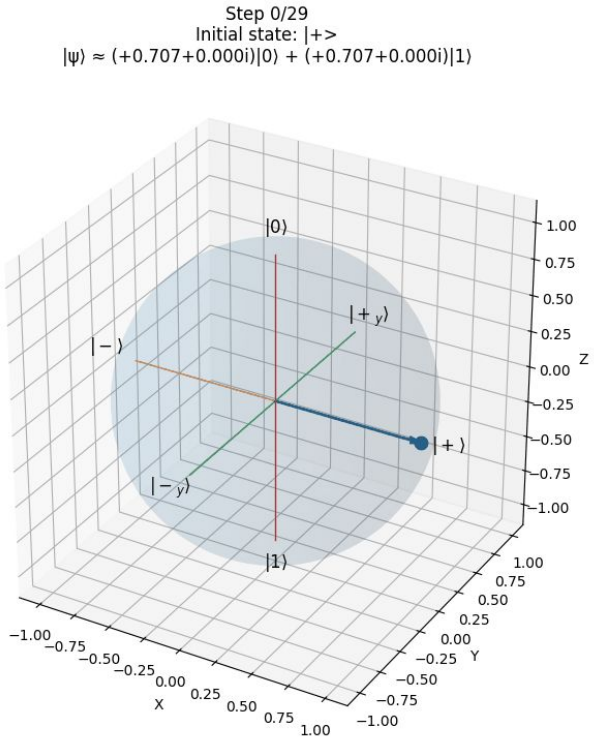
Brute forcing... tactically

a



Following this line of thought...

We can use conjugations of T's and HTH's and their inverses to generate arbitrary small angles!

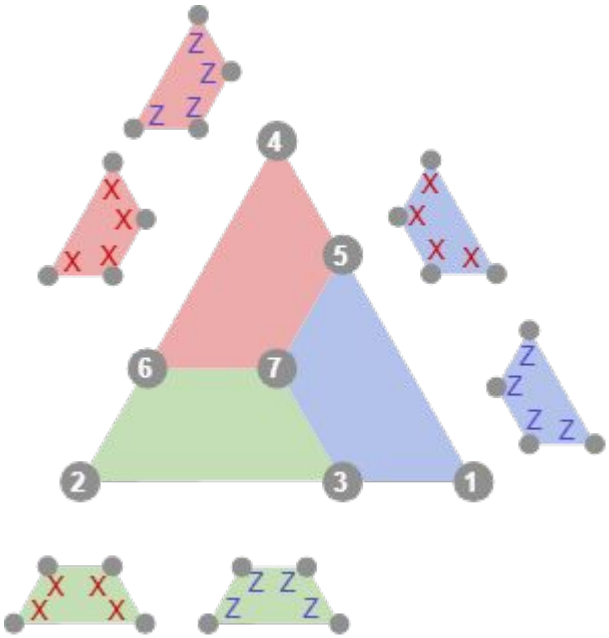


Solovay–Kitaev

Effectively involves Taylor expanding our rotations for small errors, and commutating small rotations to make smaller ones.

But… What About Errors?

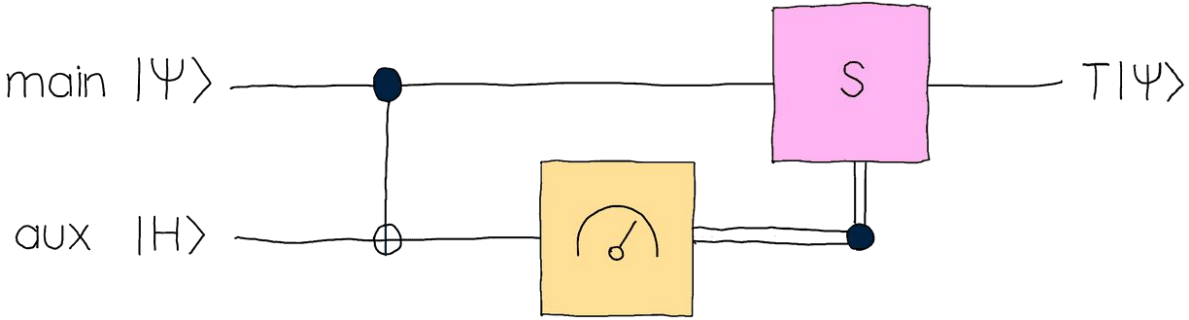
If we want to be able to correct errors, we cannot perform operations at the physical level, but need to encode our qubits using **quantum error-correcting codes** and perform computation at the **logical level**



How to apply gates to a logical qubit?

→ Transversal gates! But this is not enough…

Let the Magic Happen...



$$|T\rangle = \frac{1}{\sqrt{2}}\Big(|0\rangle + e^{i\pi/4}\,|1\rangle\Big).$$

$$\frac{1}{\sqrt{2}}\Big(\alpha\,|0\rangle + \beta e^{i\pi/4}\,|1\rangle\Big)\,|0\rangle + \frac{1}{\sqrt{2}}\Big(\alpha e^{i\pi/4}\,|0\rangle + \beta\,|1\rangle\Big)\,|1\rangle\,.$$

Transversal STAR

$$p_L(\theta) \sim |\theta| p_{ph},$$

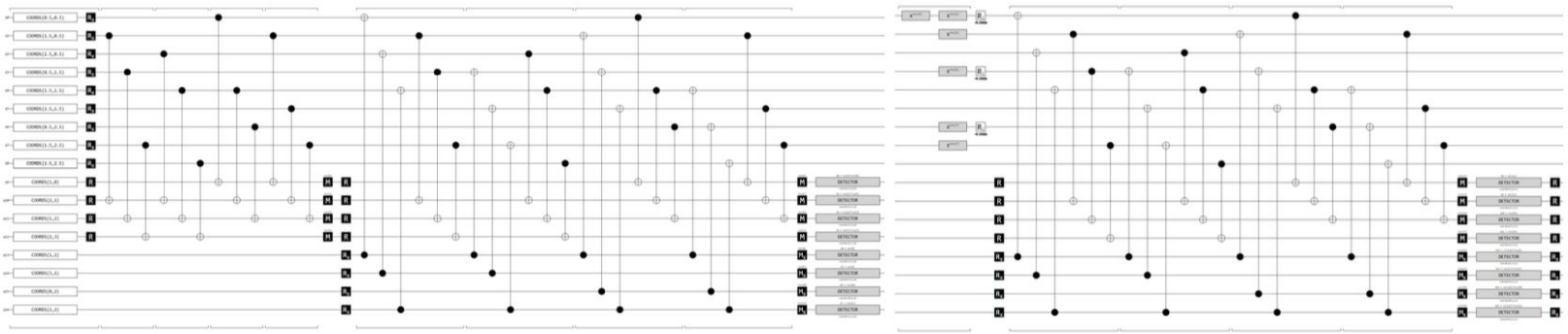
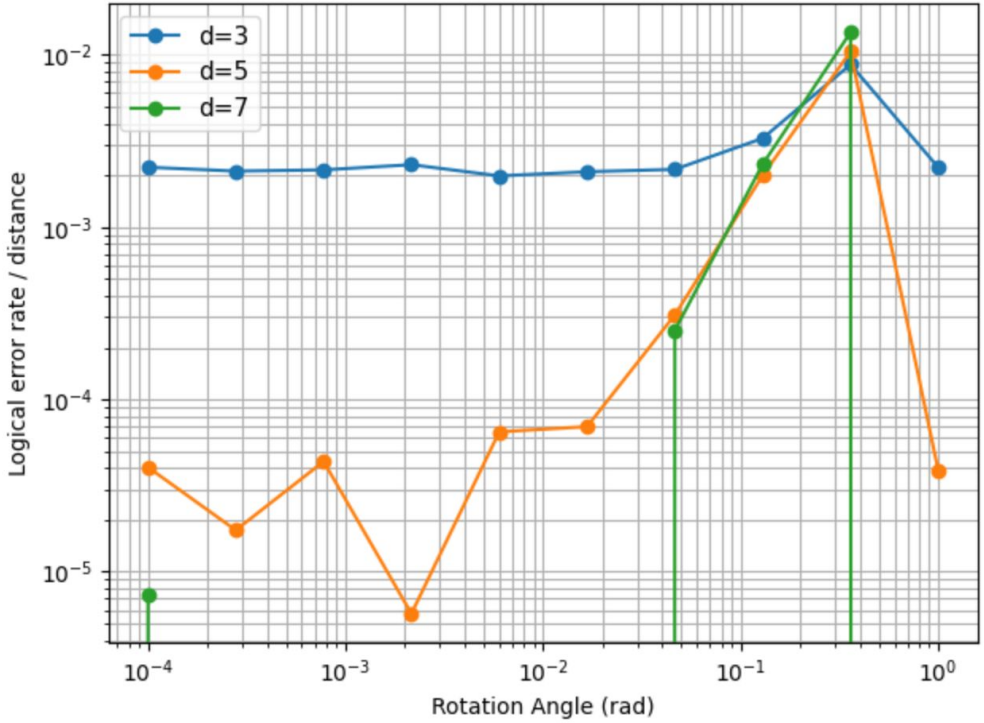


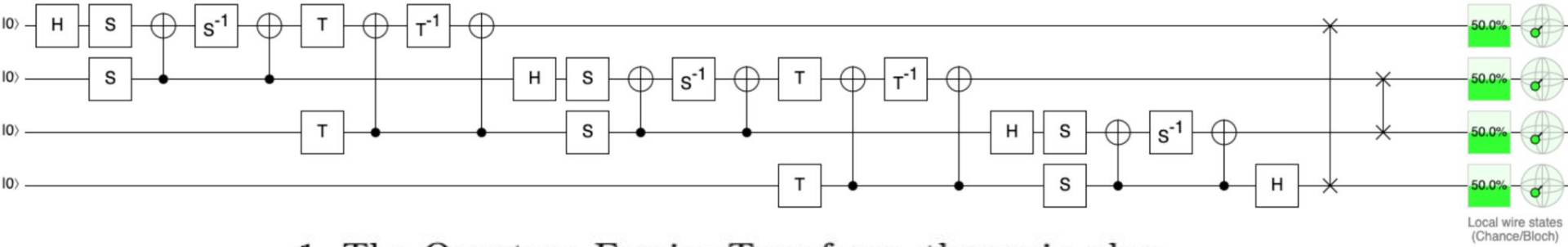
FIG. 4. First few timesteps for the STAR distance-3 circuit (without depolarizing noise for better visualization). From left to right, we see the state initialization, a couple of CNOTs followed by measurements for the syndrome extraction, three physical rotations, and finally the first out of three rounds of additional syndrome extraction.

Transversal STAR



STAR Part 4 Goal:

Demonstrate how teleported non-Clifford gates can be integrated into a larger logical circuit, and evaluate the scalability of the approach



1. The Quantum Fourier Transform, the main algorithmic engine behind Shor's algorithm for prime factorization, utilizes heavy entanglement of the logical qubits by controlled-phase rotations, where the rotation angle is $\pi/2^{k-1}$. Factoring an RSA key requires thousands of logical memory qubits, which implies the QFT would necessitate rotations with exponentially small angles such as $\pi/2^{1000}$.

Coup de grâce?

[Submitted on 15 Jun 2023]

Shor's Algorithm Does Not Factor Large Integers in the Presence of Noise

Jin-Yi Cai

We consider Shor's quantum factoring algorithm in the setting of noisy quantum gates. Under a generic model of random noise for (controlled) rotation gates, we prove that the algorithm does not factor integers of the form pq when the noise exceeds a vanishingly small level in terms of n -- the number of bits of the integer to be factored, where p and q are from a well-defined set of primes of positive density. We further prove that with probability $1 - o(1)$ over random prime pairs (p, q), Shor's factoring algorithm does not factor numbers of the form pq, with the same level of random noise present.

Craig Gidney PERMALINK

June 15, 2023 4:17 pm

It's ironic to see the "we ignored noise in the QFT" line from my paper quoted as if this was something we were worried about and ignoring, instead of something so irrelevant to the estimate that we weren't bothering.

That said, when Jin-Yi shows that you need to perform operation to a precision of epsilon ~= 8^-n where n is the number of bits in the number to factor, he's exactly right. Achieving that level of precision is why the estimate to factor a 2048 bit numbers was 20 million physical qubits instead of 4000 physical qubits. Because we are doing all operations on distance 25+ surface codes, which require 1000-2000 physical qubits per logical qubit, in order to reach the necessary tolerance of 10^-10 .

https://arxiv.org/abs/2306.10072

https://rjlipton.com/2023/06/14/a-little-noise-makes-quantum-factoring-fail/

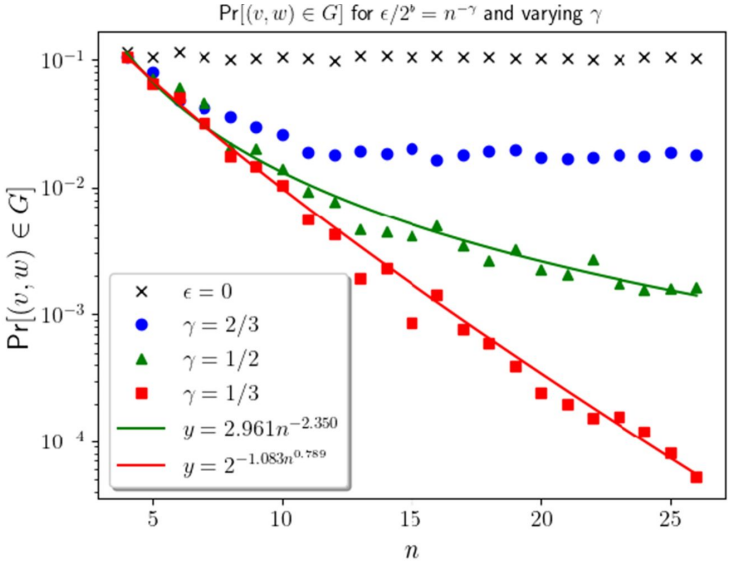


Fig. 2. Numerical simulation results: estimating the probability $\Pr[(v, w) \in G]$ for noise levels of $\frac{\epsilon}{2^b} = n^{-\gamma}$ by computing (8).

Thank you!

Thank you YQuantum for an amazingly well organized event!

Thank you QuEra for your incredible support and encouragement!

Thank you everyone for 24 hours of exhilarating learning and fun!